Disclaimer: Some illustrations in this presentation were created with AI.

STEPHAN SCHWARZ

Senior DevOps Consultant

iits-consulting GmbH



Eureg JUG

Euregio Maas-Rhine Region - Aachen, Germany

Kubernetes Workloads *under Load*

Tools, challenges and best practices

about me

Stephan Schwarz
45 years old - married, two children

Past roles:

- Network Administrator
- Systems Engineer
- Cloud Engineer
- Infrastructure Developer
- Platform Engineer

since 2017: **Kubernetes**

since 2021: **Consultant**



what about you?



a long time ago in a galaxy far far away...



What is Performance?

Performance = **Expectations are met** regarding:

- Throughput (in bytes/s, req/s or tx/s)
- Latency (e.g. p95 in ms)
- Scalability (concurrent users and factor)
- Resource consumption (% cores, GB, IOPS)
- Availability (%, number of 9s)

Expectations vary individually



Why Load Testing?

- **Stable Operations**
- Testing **Scalability**
- Ensuring **SLAs**
- Determining required **Resources**
- Optimizing **Costs**
- Uncovering **Bottlenecks**



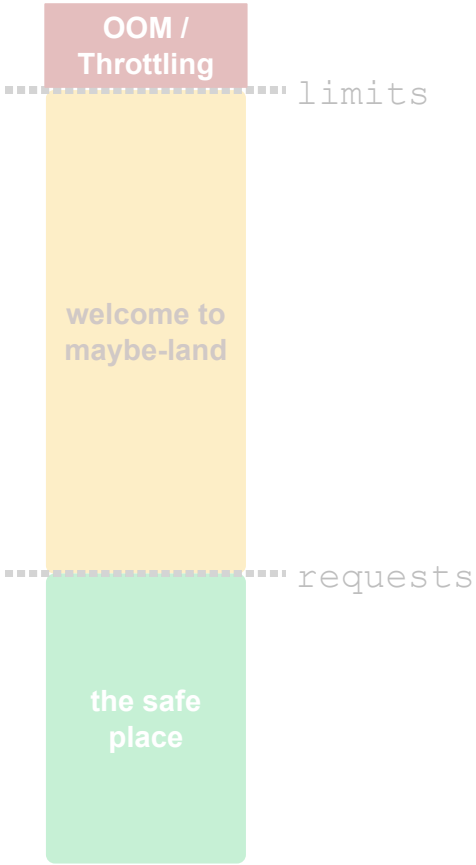
Typical Bottlenecks

CPU / RAM
Kubernetes resource requests and limits

Disk I/O & Filehandles
IOPS, max. number of file handles and threads

Network
DNS, Ingress Controller, Service Mesh

External Dependencies (DB, S3, APIs, ...)



Requirements for a Load Generator



Protocol Diversity

Support for HTTP/S, WebSockets, gRPC, MQTT, TCP, …



Test-as-Code & CI/CD

Definition of tests as code (e.g., JavaScript, Java, Python, …) and easy integration into pipelines.



Distributed Load

Native ability to scale load across multiple worker nodes or clusters.



Flexible Test Modeling

Control Virtual Users (VUs), requests/s, ramp-up/down & scenarios (e.g. via HAR files)



Metrics

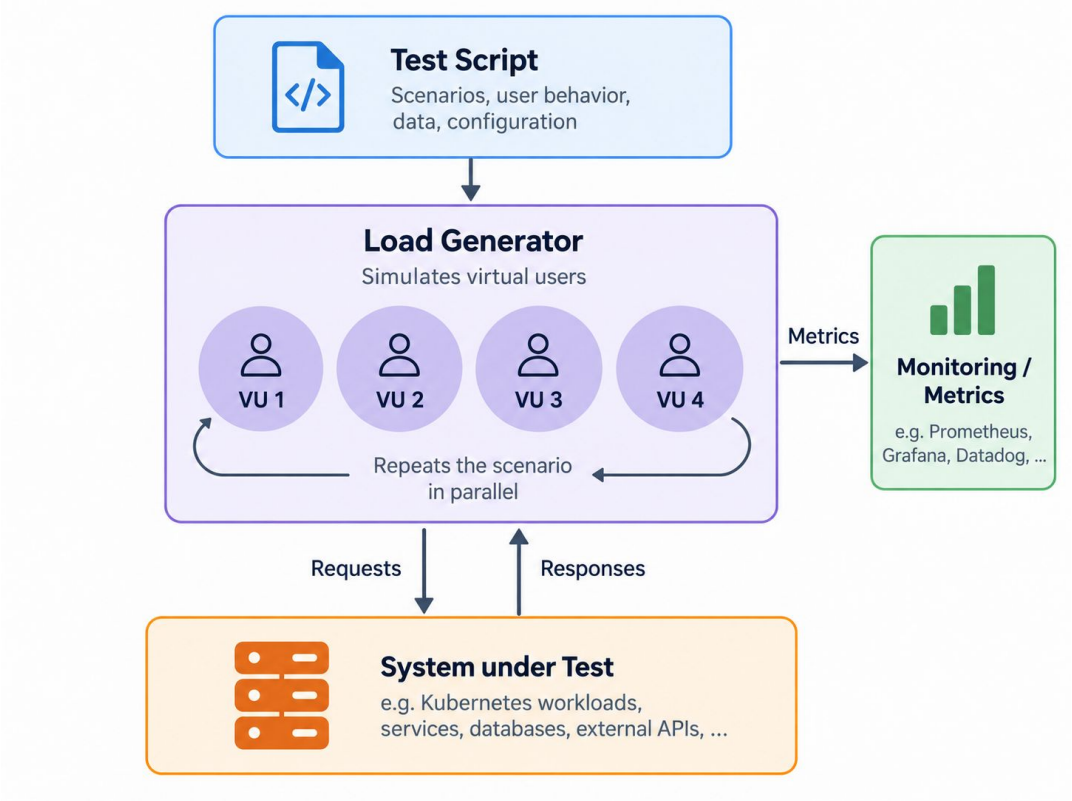
Streaming of live data (Prometheus, InfluxDB, …) and detailed reports.



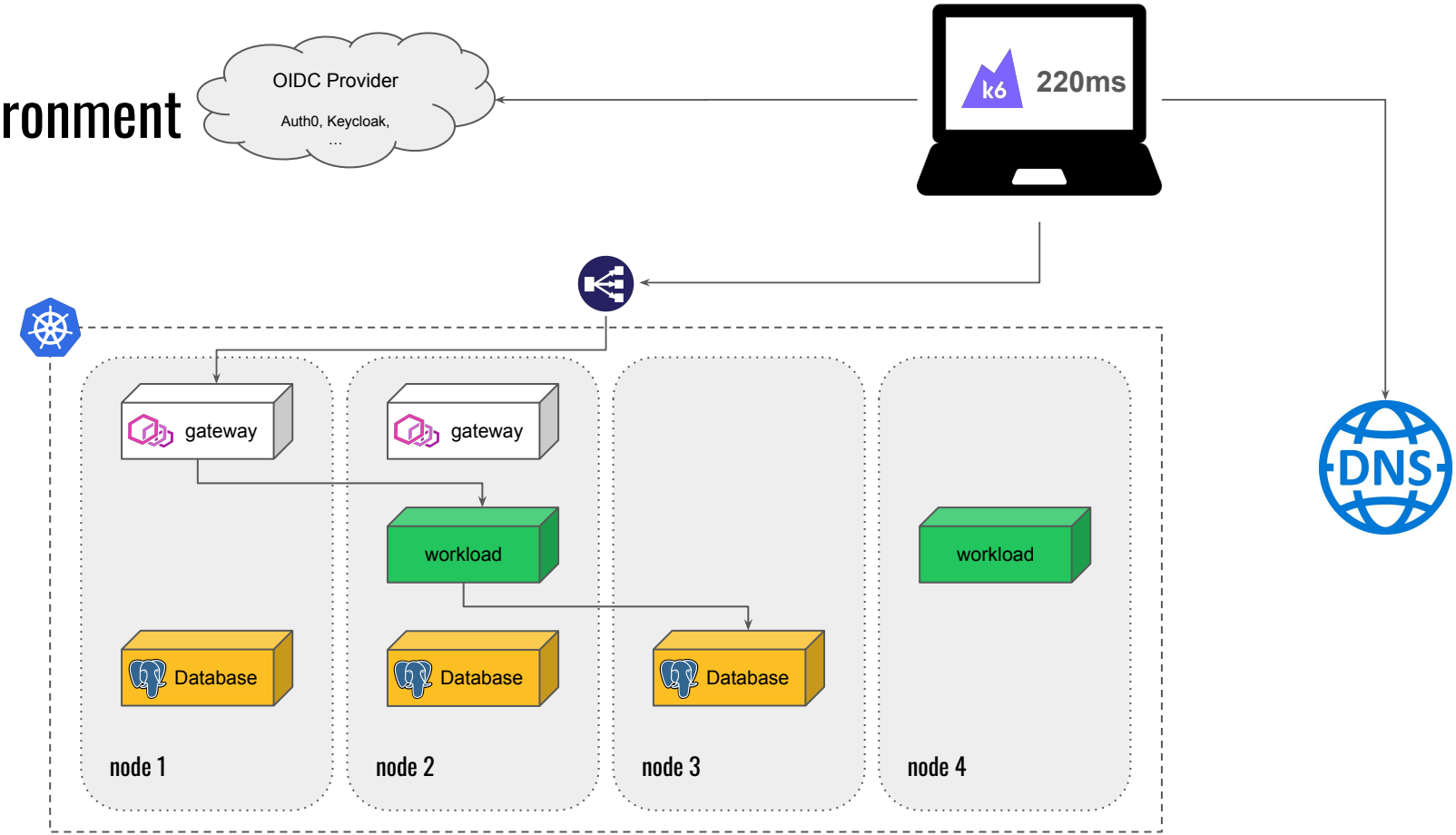
Extensibility

Easy loading of third-party libraries or custom protocols via plugins.

How Does a Load Generator Work?

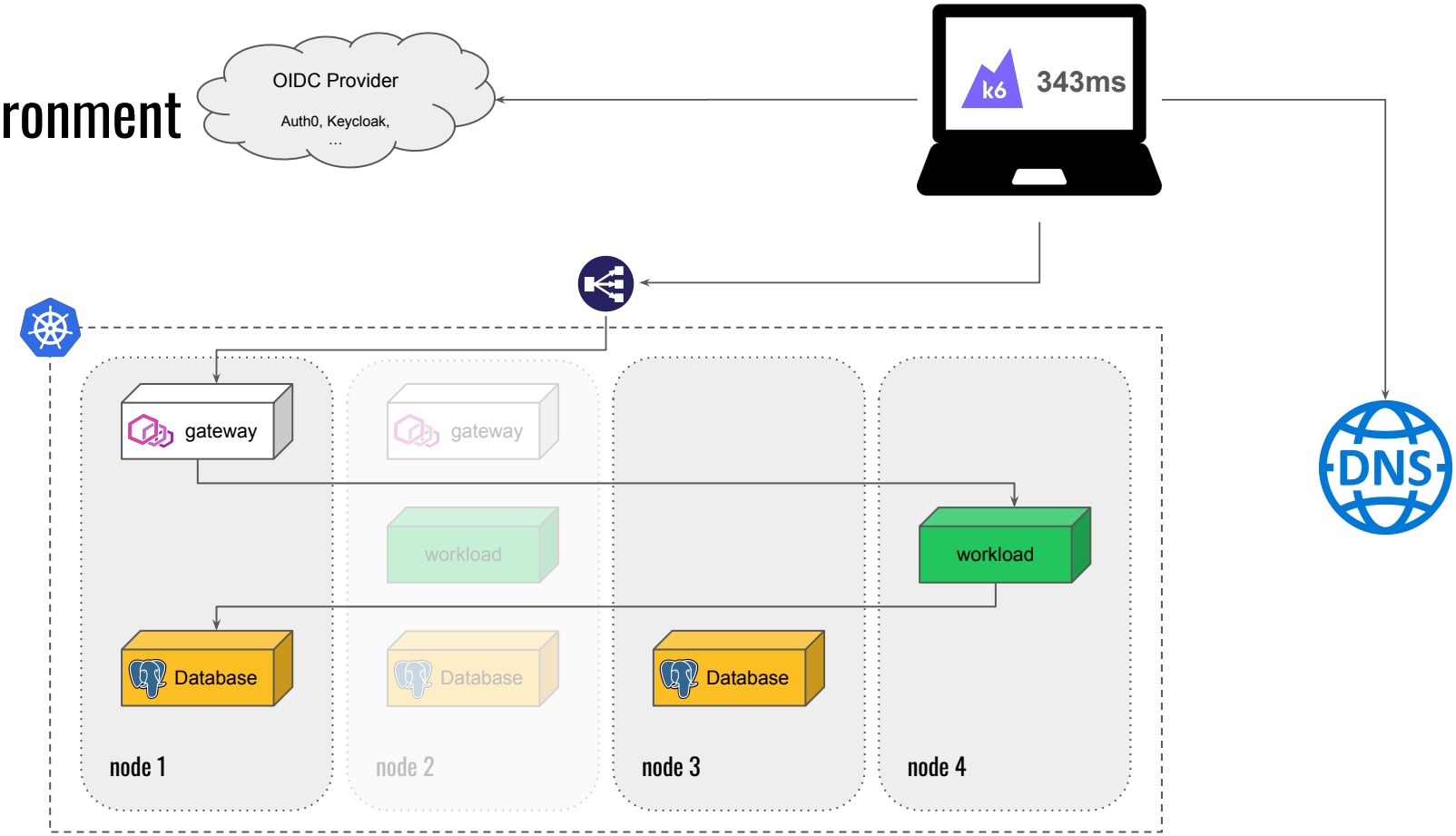


Test environment



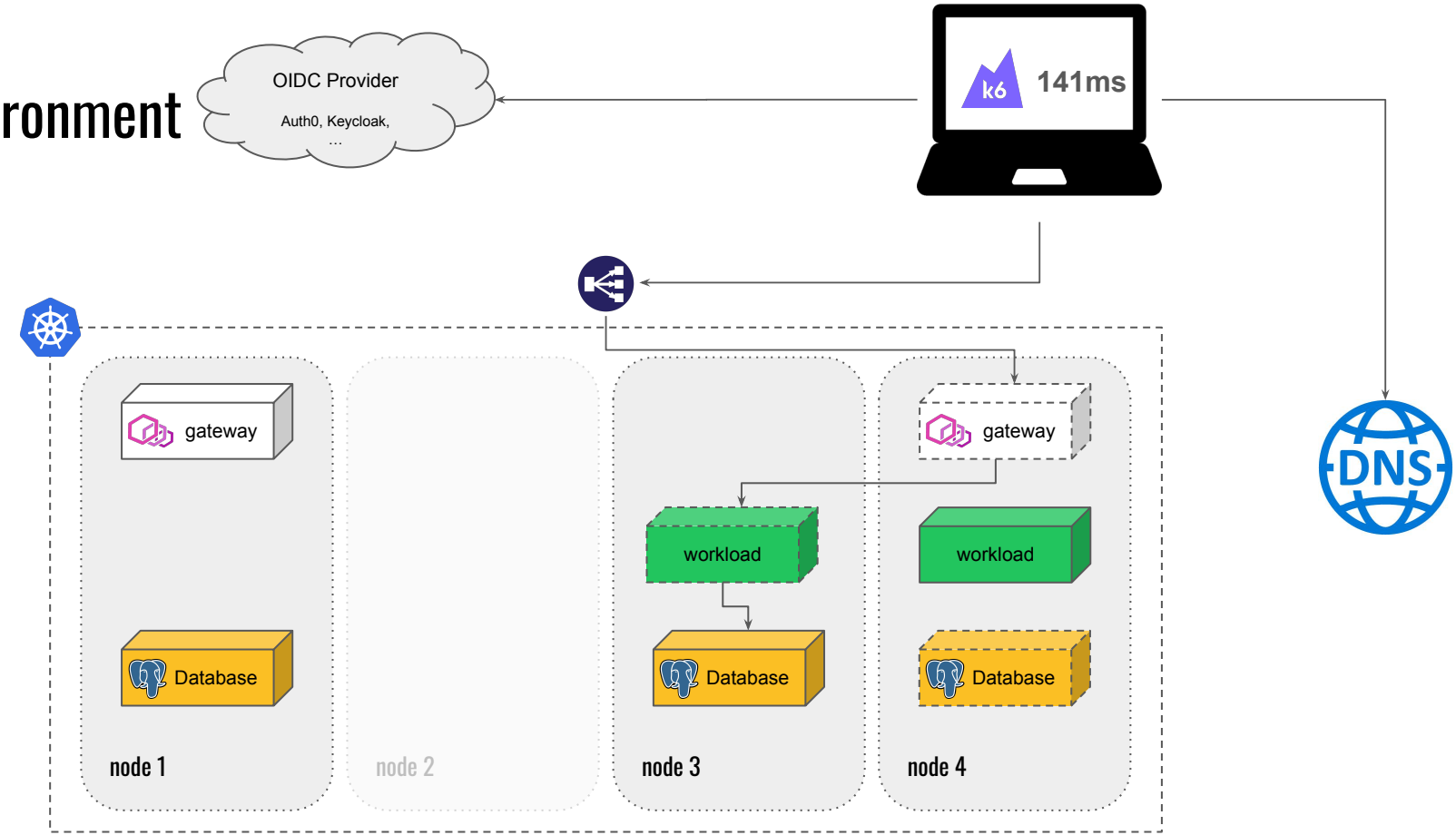


Test environment





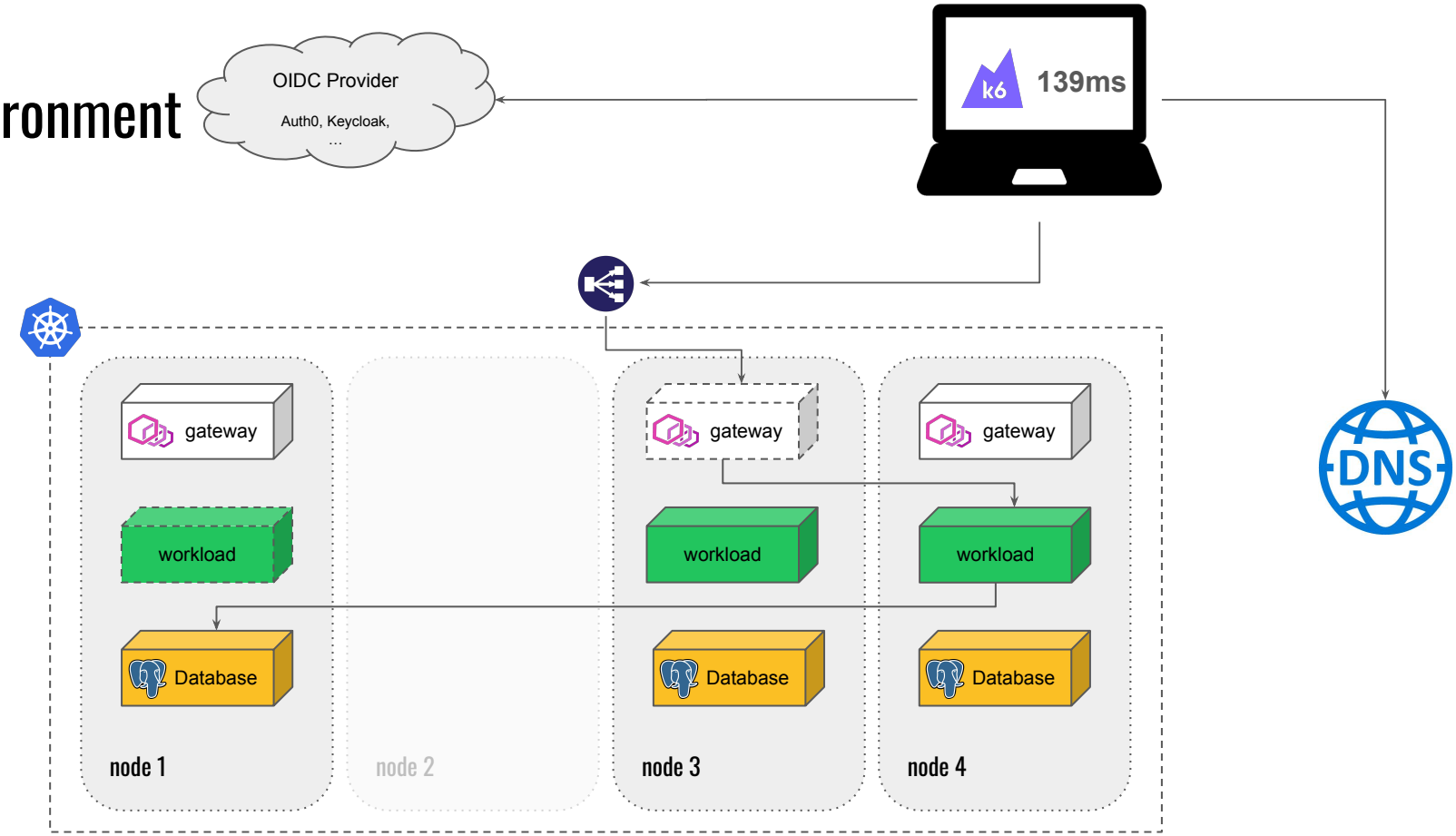
Test environment





Re-scheduling

Test environment

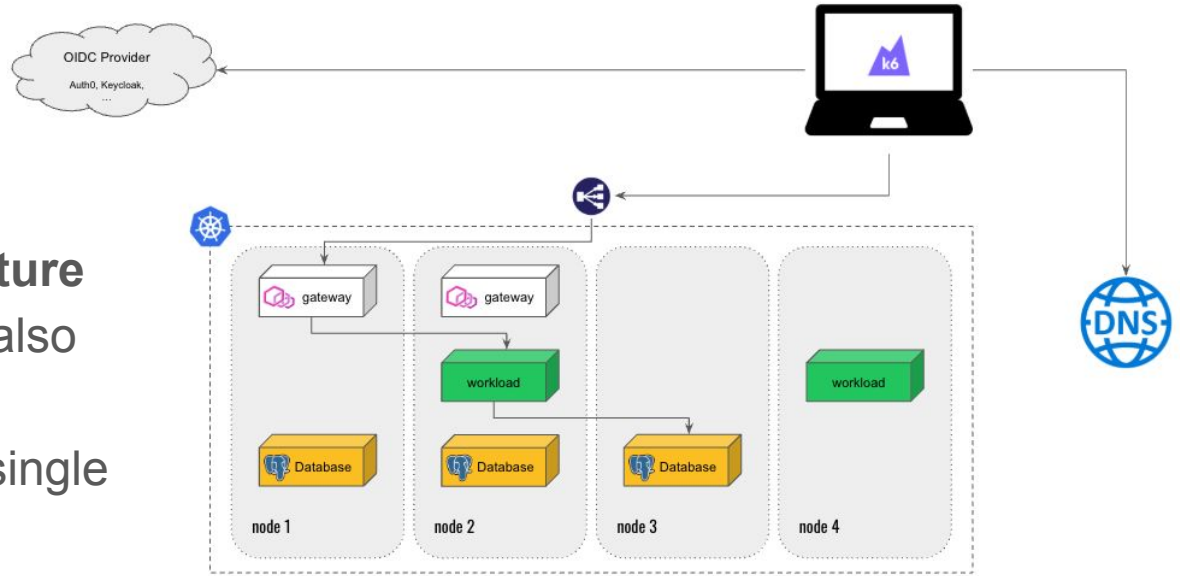




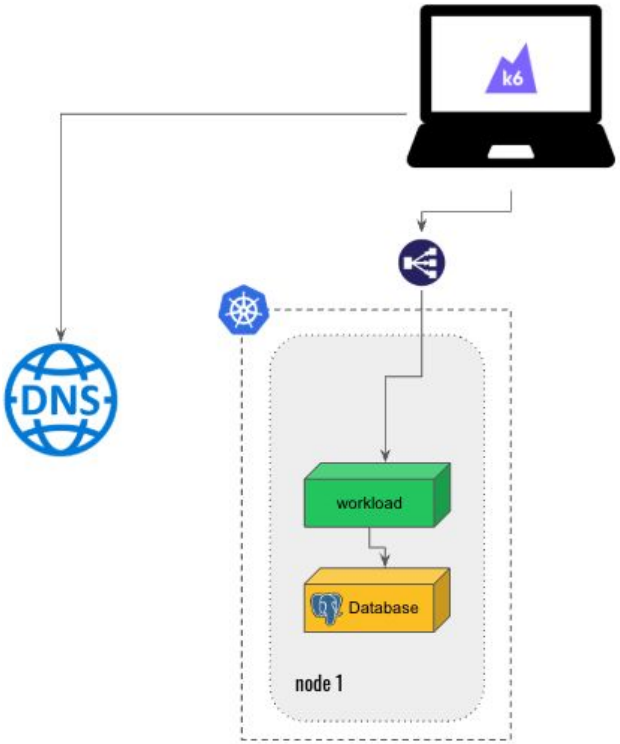
Test environment

End-to-end

- **Realistic**
- **Customer perspective**
- Tests complete **infrastructure**
- **Dependent services** are also tested
- **No targeted testing** of a single component



Test environment

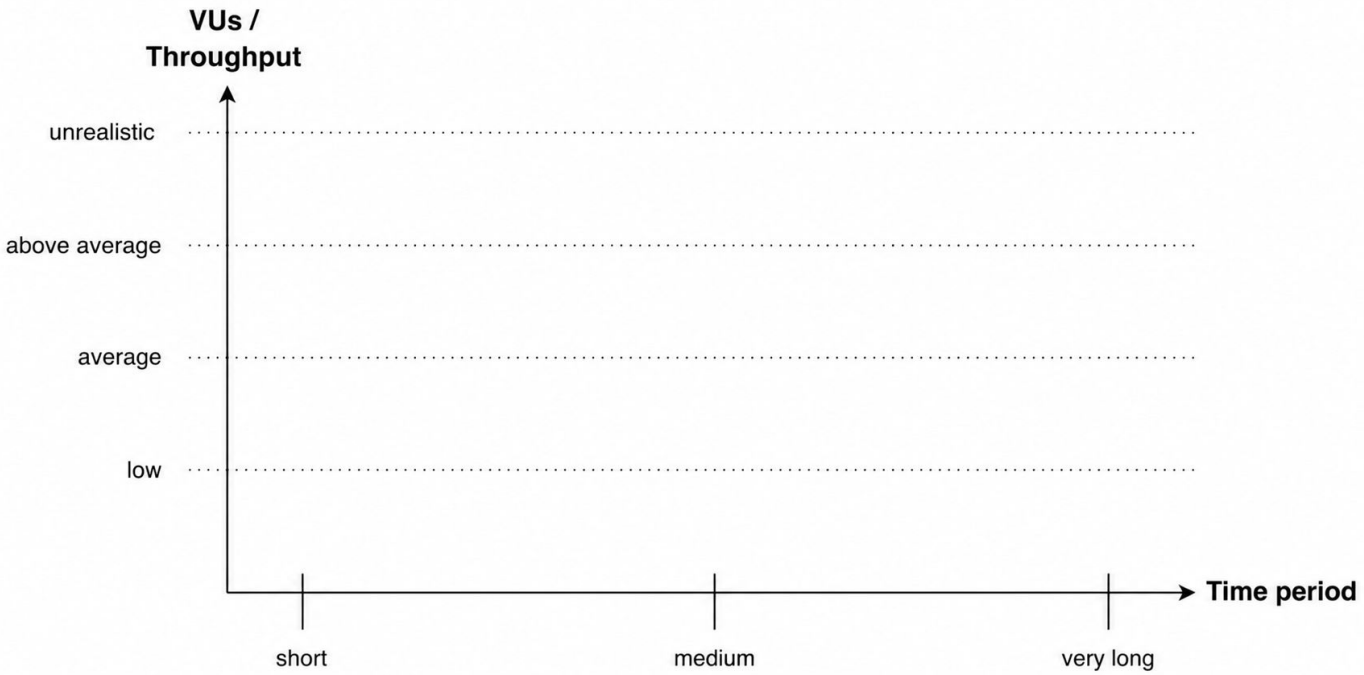


Lab Environment

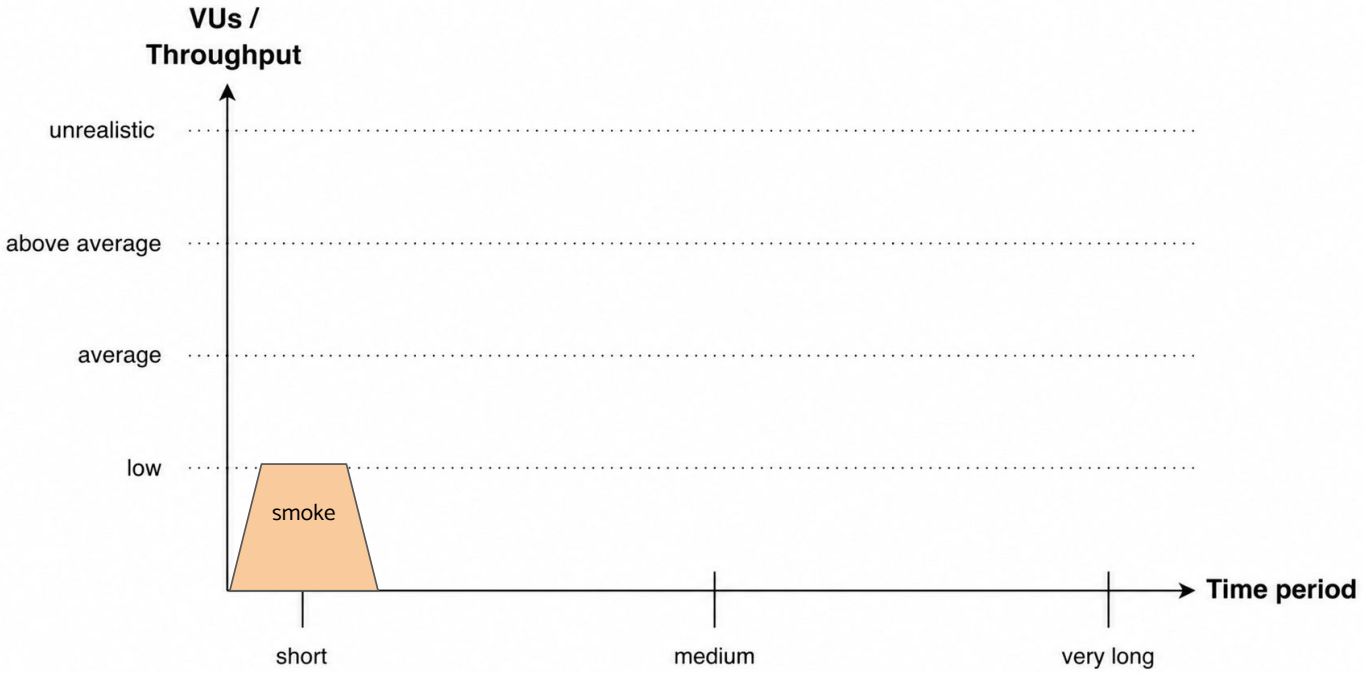
- e.g., **Single-Pod** perspective
- Determining **resource usage**
- Experimenting with values for **resource requests** and **limits**
- Only **necessary dependencies** are tested
- e.g., **Single-Node** cluster **without autoscaling**



Types of Load Tests



Smoke Test

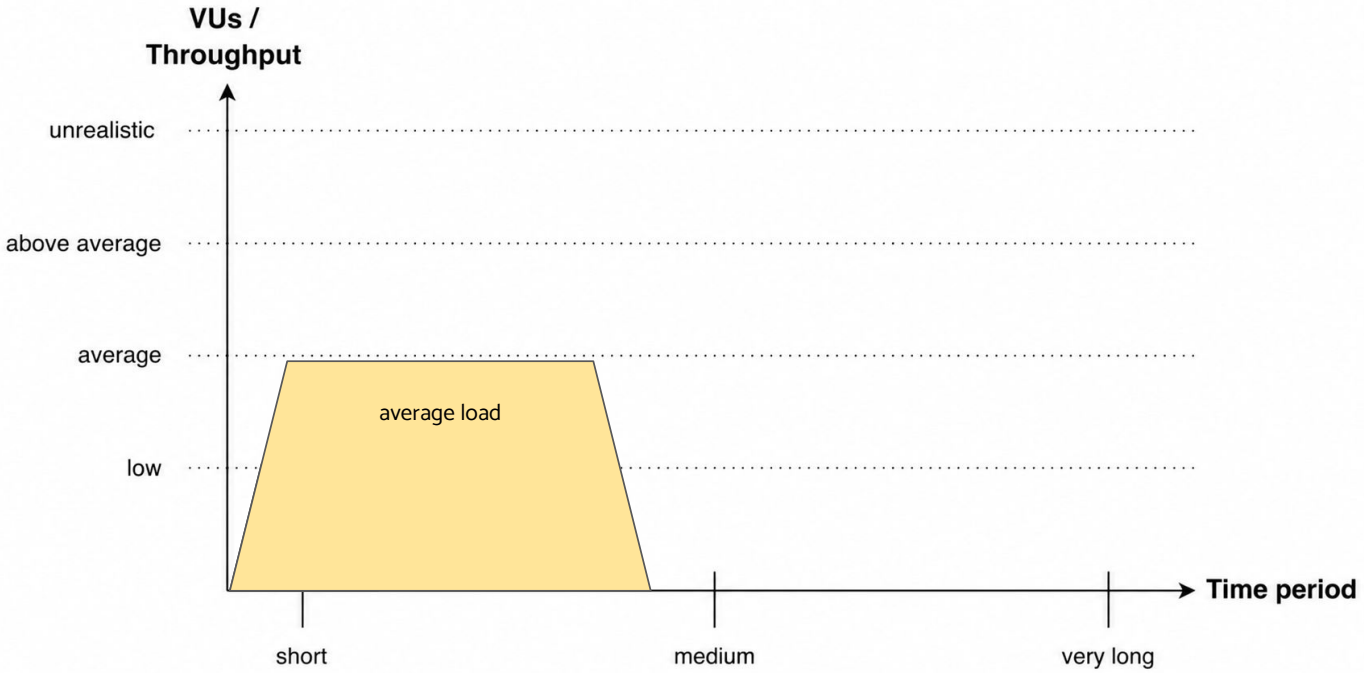


Minimal load test

Validating a test scenario

Integration into pipeline
to uncover regressions

Average Load Test



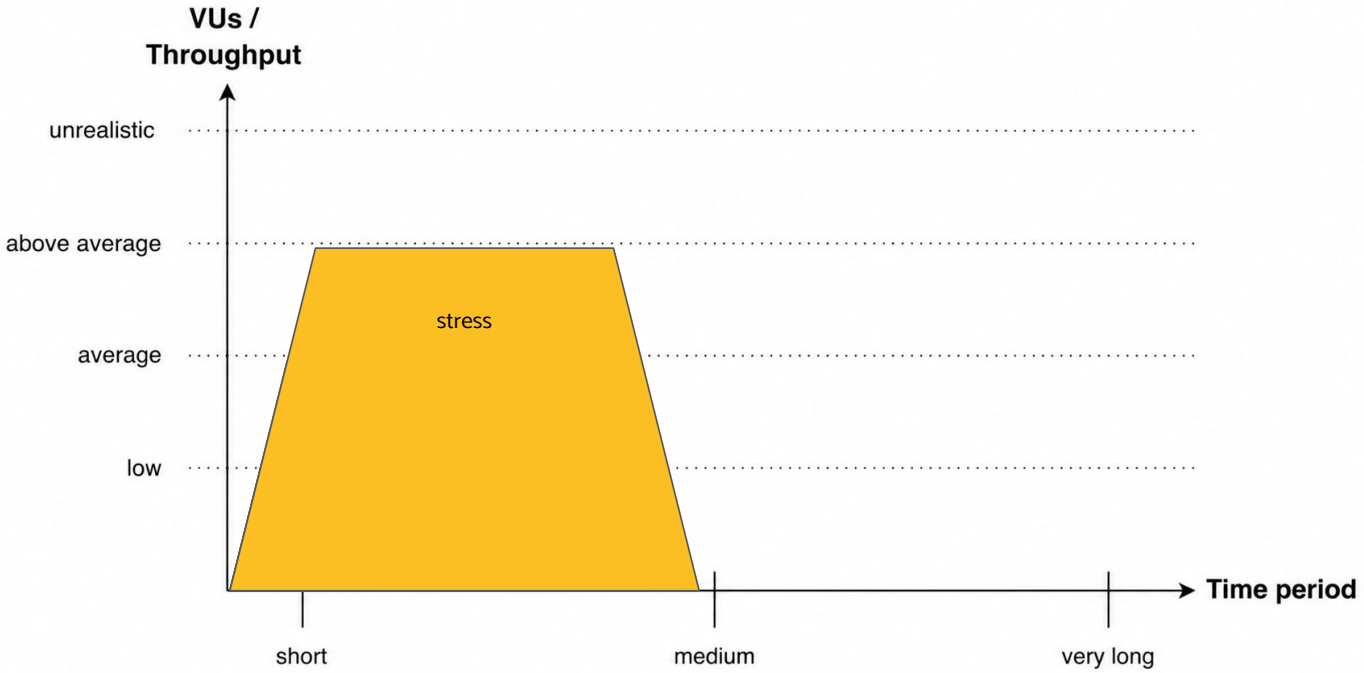
Everyday load

Determining the baseline

- CPU usage
- RAM usage
- Response time

Stable operation (SLA)

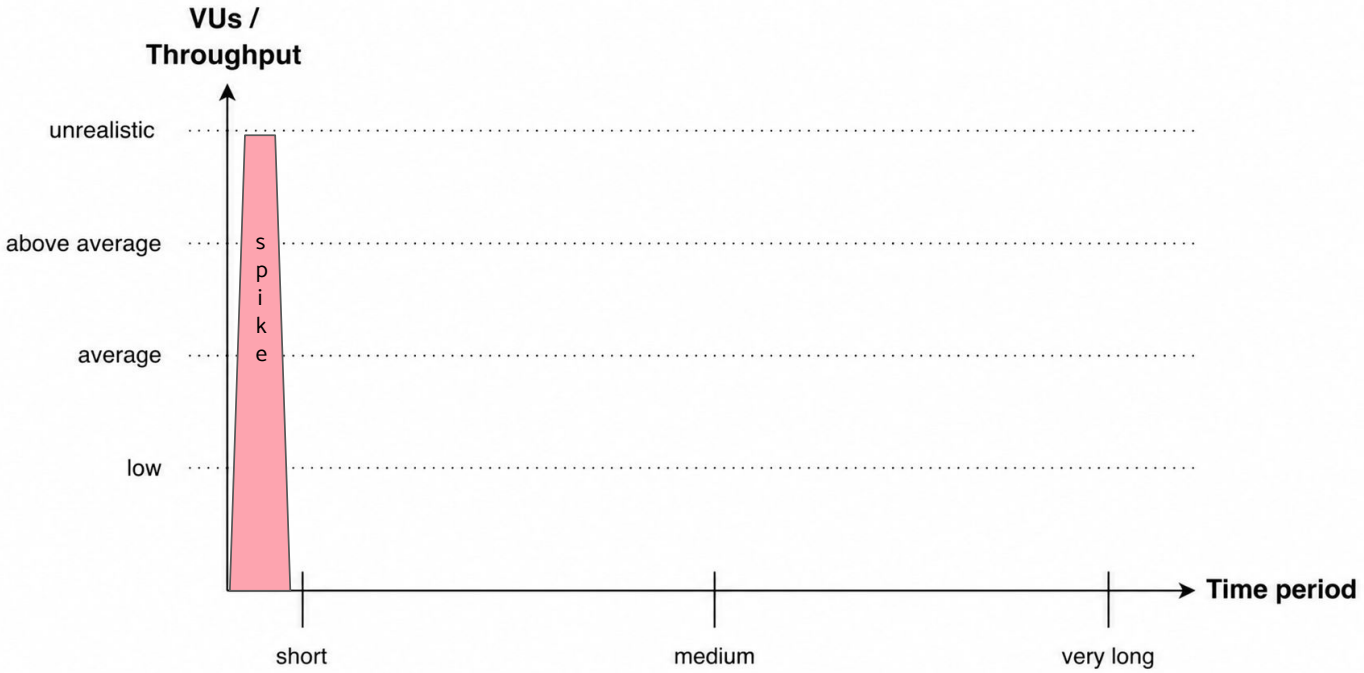
Stress Test



Black Friday Test (1/2)

Test des autoscalings
und der Resilienz

Spike Test



Black Friday Test (2/2)

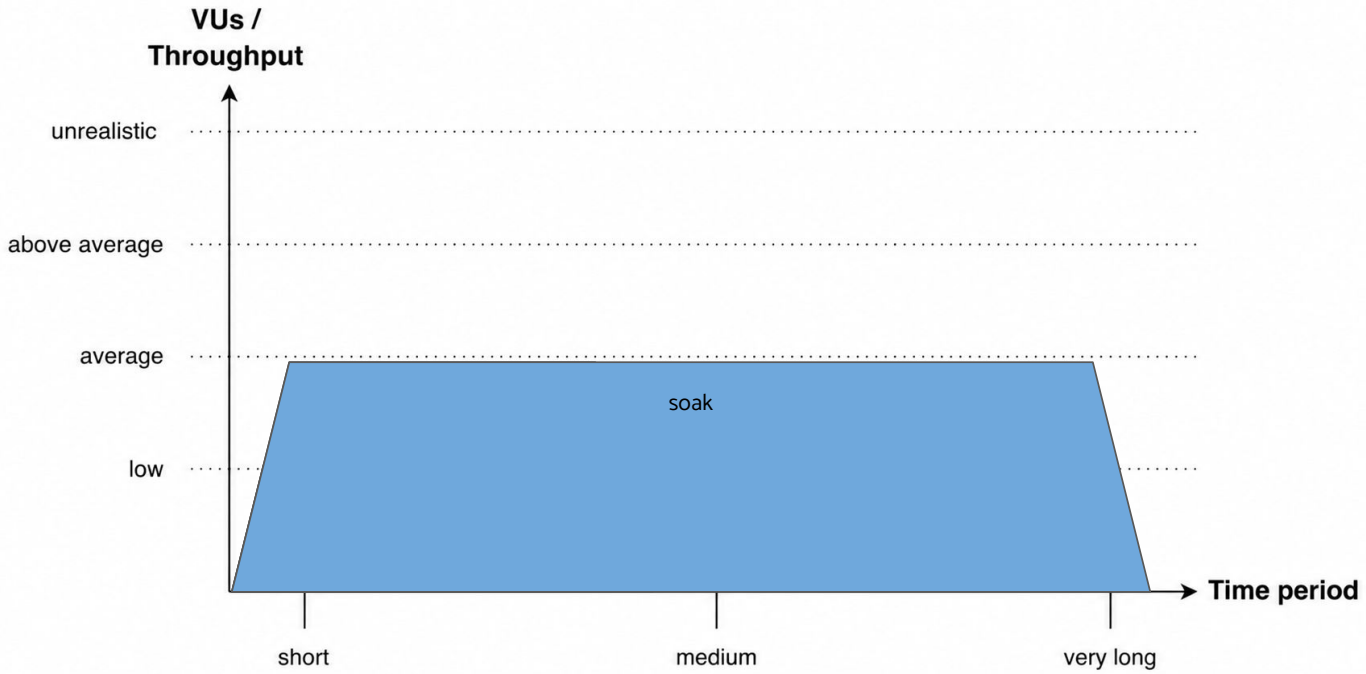
Scale up/down

- HPA
- Cluster autoscaler

Resilience

- Outage?
- How long? (MTTR)
- Blast Radius

Soak Test



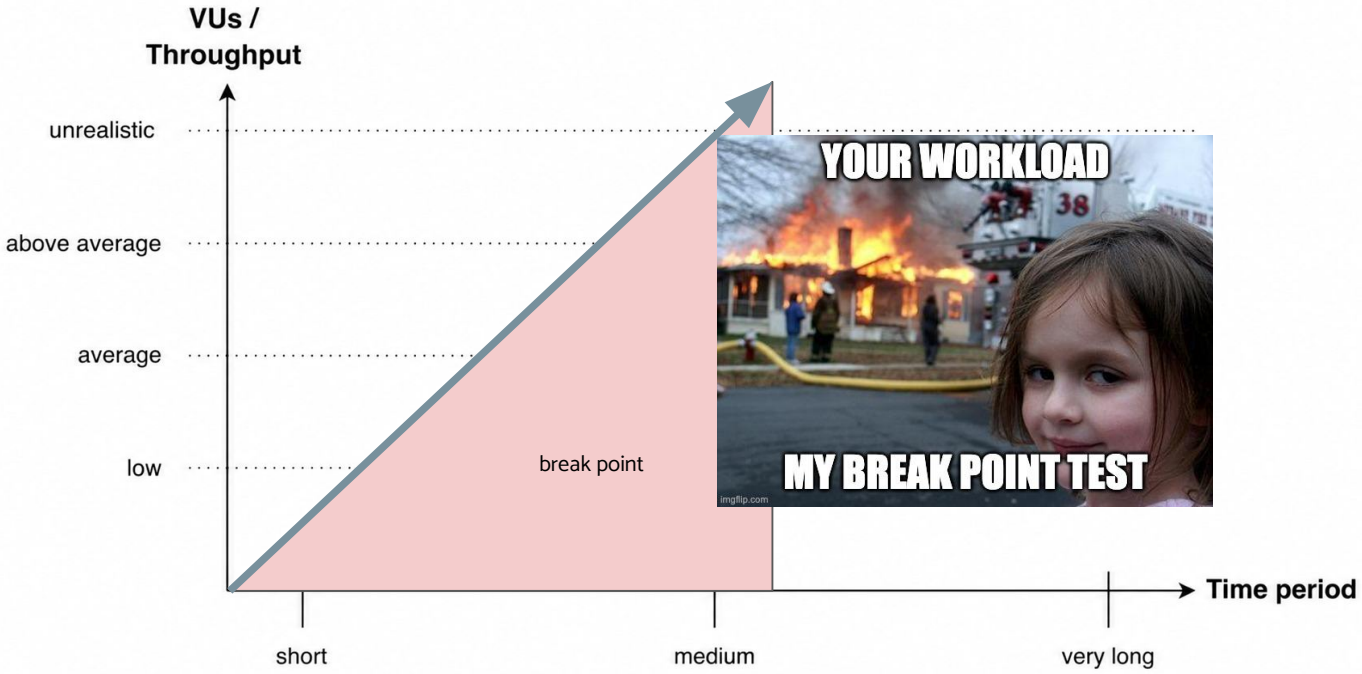
Long-term test (e.g. 24h)
Identifying long-term effects:

- Memory Leaks
- Connection Pool Exhaustion
- Performance Degradation
- Disks filling up (Logs / Database)

Behavior during maintenance tasks:

- Upgrades
- Migrations

Break Point Test



Increase until collapse

Determine:

- max. capacity
- Bottlenecks

What is the right test / the right test environment for me?



It depends…

Which individual questions need to be answered?



Combination

A single test rarely answers all relevant questions directly



Evolutionary

Over time, your questions will change.

Test Example

```
import { check, sleep } from "k6";
import http from "k6/http";

export let options = {
  stages: [
    { duration: "30s", target: 10 },     // ramp up
    { duration: "3m",  target: 10 },     // main
    { duration: "30s", target: 0 },      // ramp down
  ]
};

export default function() {
  let res = http.get("https://iits-consulting.de");
  check(res, {
    "is status 200": (r) => r.status === 200
  });
  sleep(1);
};
```

```
~/Projekte/performance-talk
☺  k6 cloud run ./basic-load.js
```




```
WARN[0000] No projectID specified, using default project of the https://stearz.grafana.net stack

     execution: cloud
        script: ./basic-load.js
        output: https://stearz.grafana.net/a/k6-app/runs/8523080

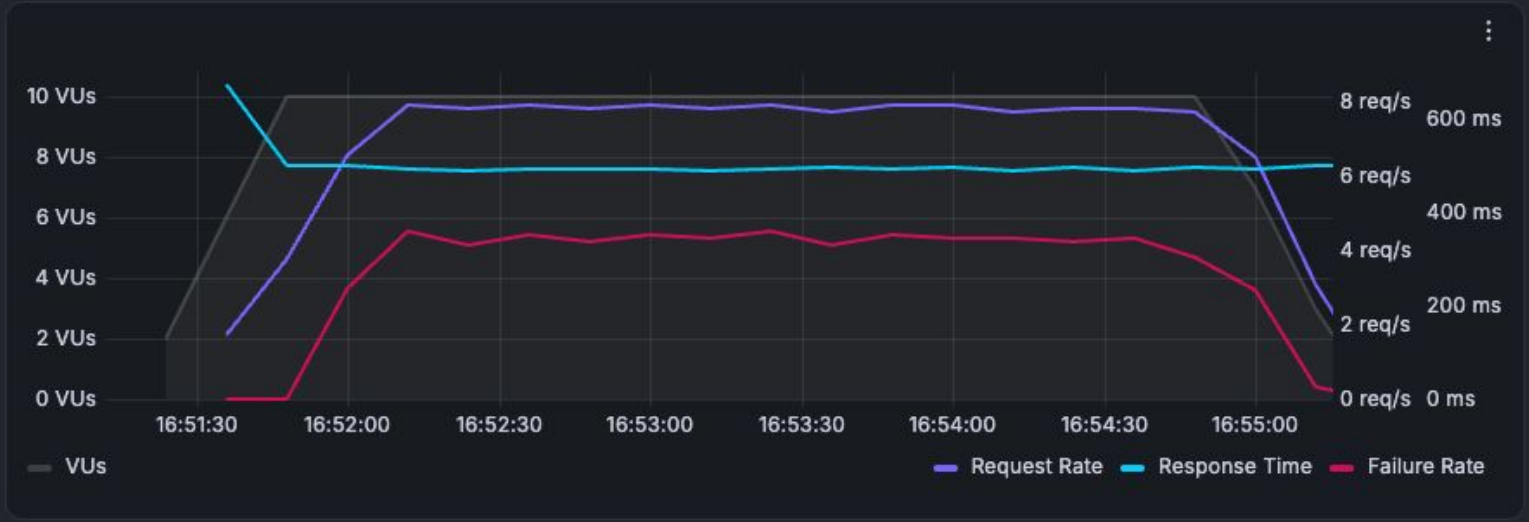
     scenarios: (100.00%) 1 scenario, 10 max VUs, 4m30s max duration (incl. graceful stop):
              * default: Up to 10 looping VUs for 4m0s over 3 stages (gracefulRampDown: 30s, gracefulStop: 30s)


Run    [=======>------------------------------] 0m57.7s/4m30s
```

PERFORMANCE OVERVIEW

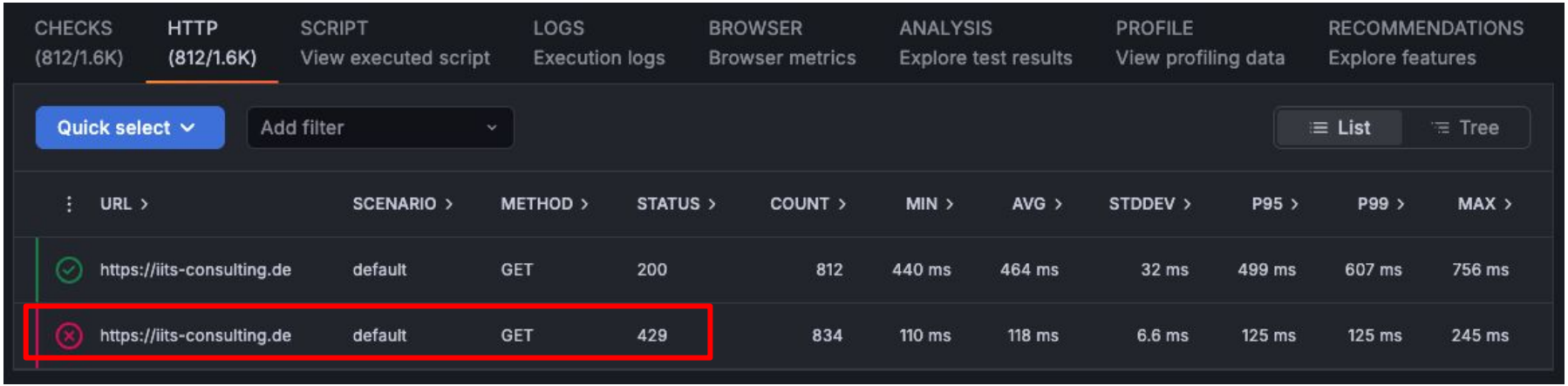
The 95th percentile response time of the system being tested was 495 ms, and 1 646 requests were made with 834 failures at an average rate of 6.8 requests/second.

REQUESTS MADE	HTTP FAILURES	PEAK RPS	P95 RESPONSE TIME
1.6K reqs	834 reqs	7.92 reqs/s	495 ms



SCENARIOS (0) SHOW

No scenario was configured in this test. Check out how to create one

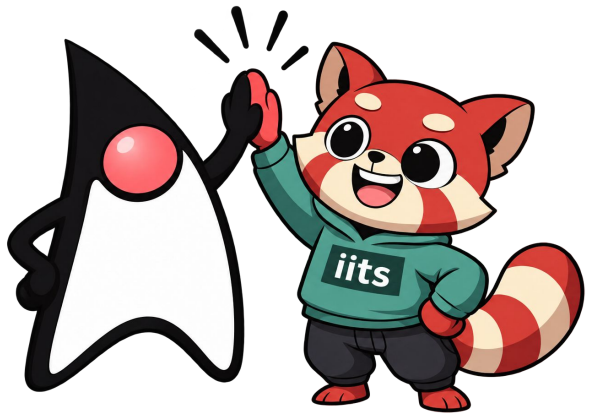


HTTP/1.1 429 Too Many Requests

Lessons Learned (1/3)

Access to monitoring systems required:

- Alerts
- Metrics
- Logs
- Traces



Lessons Learned (2/3)

Adjust Test Environment

Sometimes test results require adjusting the test environment while testing

Self-Managed Kubernetes

If you manage Kubernetes yourself: Set `kubeReserved` and `systemReserved` before any testing

I/O Caching

Leave some free RAM on the node for I/O Caching

It’s still hardware (somewhere)

Sometimes 2 Cores < 1 Core (Node types matter)

Lessons Learned (3/3)



Communication

Coordinate load tests in advance with relevant departments and service providers.

+1

Maintain Focus

Never make more than one change at a time.



Documentation

Document every change and its corresponding result.



Reproducibility

Repeat tests to rule out random external factors.



Root Cause Analysis

If something breaks, find the cause(s), document, and then repair



Realistic Expectations

Things will and must break.
This is the way.

Why load testing is unpopular...

Team Expertise

Requires cross-team expertise and broad domain knowledge.

Low Momentum

Motivation of one team is rarely enough to activate other teams.

Local Illusion

"Works on my machine"

Effort

Execution costs time and money.

More Than Just Tech

Load testing is not exclusively a technical project.

Not Trivial

Complex scenarios require planning and analysis.

Why load tests are unpopular... - *...but necessary!*

**“If you don't test yourself,
you let your customers test.”**



Do or do not.
There is no try.





Thank you

stephan.schwarz@iits-consulting.de